

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №3
по дисциплине
«Функциональное программирование»
Вариант 5

Студент,
группы 5130201/30102

_____ Санько В. В.

Преподаватель,
к. т. н., доц.

_____ Моторин Д. Е.

« _____ » _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
Постановка задачи	4
1 Особенности реализации	5
1.1 Функторы и аппликативные функторы	5
1.2 Реализация TransactionMonitor как newtype	5
1.3 Реализация Functor для TransactionMonitor	5
1.4 Реализация Applicative для TransactionMonitor	5
2 Результаты работы программы	7
2.1 Тестирование транзакций	7
2.2 Демонстрация FinancialSummary	7
2.3 Демонстрация Monoid	8
Заключение	9
Список литературы	11
Приложение. Исходный код	12

Введение

Финансовая модель транзакций — это система учета и классификации денежных операций, которая позволяет анализировать финансовые потоки. В основе такой модели лежит формализованное представление транзакций с четко определенными атрибутами: идентификатором, суммой, типом операции и описанием.

Отличительной особенностью финансовой модели является разделение транзакций по типам операций. Доходы - это поступления денежных средств, расходы — затраты, а переводы — перемещения между счетами.

Транзакции могут быть классифицированы разными способами, используя механизмы фильтрации и разбиения на группы. Например, логика проверки пороговой стоимости позволяет выделять значимые средства. С другой стороны, при совместном использовании нескольких критериев возможна многомерная классификация транзакций, что позволяет получить максимум данных из информации и, соответственно, увеличить степень контроля.

В данной работе представлена расширенная версия финансовой модели. Введены типы `FinancialSummary` для статистической сводки и `TransactionMonitor` а для расширенного анализа транзакций, и реализовали для них представителей указанных классов типов.

Постановка задачи

Цель работы – разработать расширение системы финансового мониторинга, включающее:

- Финансовая статистику;
- Система мониторинга транзакций;
- Реализовать проект в Stack.

1 Особенности реализации

1.1 Функторы и аппликативные функторы

Реализация Semigroup и Monoid для FinancialSummary:

Листинг 1: Определение типов данных для финансовых транзакций

```
1 instance Semigroup FinancialSummary where
2   (FinancialSummary inc1 exp1 cnt1 max1) <> (FinancialSummary inc2 exp2 cnt2 max2) =
3   FinancialSummary
4   { totalIncome = inc1 + inc2
5     , totalExpense = exp1 + exp2
6     , transactionCount = cnt1 + cnt2
7     , maxTransaction = case (max1, max2) of
8       (Nothing, Nothing) -> Nothing
9       (Just t1, Nothing) -> Just t1
10      (Nothing, Just t2) -> Just t2
11      (Just t1, Just t2) ->
12        if amount t1 >= amount t2
13        then Just t1
14        else Just t2
15   }
```

Использование Maybe Transaction для представления максимальной транзакции позволяет корректно обрабатывать отсутствие данных. Нейтральный элемент mempty содержит нулевые значения и Nothing для максимальной транзакции.

1.2 Реализация TransactionMonitor как newtype

Функция Конструктор:

Листинг 2: Функция проверки порогового значения

```
1 newtype TransactionMonitor a = TransactionMonitor
2   { runMonitor :: Transaction -> (Bool, a)
3   }
```

Паттерн позволяет отделить логику мониторинга от её выполнения.

1.3 Реализация Functor для TransactionMonitor

Интерфейс:

Листинг 3: Функция классификации транзакций

```
1 instance Functor TransactionMonitor where
2   fmap f (TransactionMonitor monitor) = TransactionMonitor $ \t ->
3   let (needsAttention, result) = monitor t
4   in (needsAttention, f result)
```

Функция fmap применяет преобразование только к значению a, не изменяя флаг Bool.

1.4 Реализация Applicative для TransactionMonitor

Создает монитор, который никогда не требует внимания, а также возвращает константное значение для любой транзакции.

Листинг 4: Вспомогательные функции-предикаты

```
1 pure x = TransactionMonitor $ \_ -> (False, x)
   <*> (апликация):
```

Листинг 5: Вспомогательные функции-предикаты

```
1 (TransactionMonitor f) <*> (TransactionMonitor g) =
2 TransactionMonitor $ \t ->
3 let (needsAttention1, func) = f t
4     (needsAttention2, value) = g t
5 in (needsAttention1 || needsAttention2, func value)
```

Это означает: если хотя бы один монитор требует внимания, результирующий монитор также требует внимания, при этом значения вычисляются независимо для каждого монитора.

2 Результаты работы программы

2.1 Тестирование транзакций

Для проверки работоспособности подготовлен набор тестовых транзакций Рис. 1.

```
ghci> let normalTransaction = Transaction 1 500.0 Expense "Покупка продуктов" "Основной счет"
ghci> let largeTransaction = Transaction 2 1500.0 Income "Зарплата" "Основной счет"
ghci> let suspiciousTransaction = Transaction 3 (-100.0) Expense "Возврат?" "Основной счет"
ghci> complexMonitoringSystem normalTransaction
[Nothing,Nothing,Nothing]
ghci> complexMonitoringSystem largeTransaction
[Just "10421053104010521040105310401045: 1041108610831100109610721103 1090108810721085107910721082109410801103 10851072 10891091108410841091 1500.0",Nothing,Just "10421053104010521040105310401045: 1050108810721085107910721082109410801103 10471072108810871083107210901072 10851072 10891091108410841091 1500.0"]
ghci> complexMonitoringSystem suspiciousTransaction
[Nothing,Just "10421053104010521040105310401045: 10851086107310861079108810801090107710831108108510721103 1090108810731085107910721082109410801103 10421086107910741088107210907 10851072 10891091108410841091 -100.0",Just "10421053104010521040105310401045: 1050108810721085107910721082109410801103 10421086107910741088107210907 10851072 10891091108410841091 -100.0"]
```

Рис. 1: Тестирование нормальной большой и подозрительной транзакции

2.2 Демонстрация FinancialSummary

На рисунке ниже Рис. 2 отображено, что программа скомбинировала сразу несколько данных.

```
ghci> let transactions = [normalTransaction, largeTransaction, suspiciousTransaction]
ghci> let summary = transactionsToSummary transactions
ghci> show (totalIncome summary)
"1500.0"
ghci> show (totalExpense summary)
"400.0"
ghci> show (transactionCount summary)
"3"
```

Рис. 2: Пример использования функции

2.3 Демонстрация Monoid

Агрегация финансовой статистики: от отдельных транзакций к сводному отчету показан на Рис. 3.

```
ghci> let summary1 = transactionToSummary normalTransaction
ghci> let summary2 = transactionToSummary largeTransaction
ghci> let combined = summary1 <> summary2
ghci> show combined
"FinancialSummary (totalIncome = 1500.0, totalExpense = 500.0, transactionCount = 2, maxTransaction = Just (Transaction {transactionId = 2, amount = 1500.0, transactionType = Income, description = \"10421053104010521040105310401045: 1041108610831100109610721103 1090108810721085107910721082109410801103 10851072 10891091108410841091 1500.0\"}), account = \"10541089108510861074108510861081 1089109510771090\"))"
```

Рис. 3: Примеры использования функций на разных значениях

Система работает правильно: мониторы находят подозрительные транзакции, а статистика корректно суммируется. Все функции выполняют свои задачи без ошибок.

Заключение

В ходе работы была реализована система мониторинга финансовых транзакций на Haskell, демонстрирующая применение функциональных абстракций. Созданы два ключевых компонента: `FinancialSummary` с поддержкой моноидных операций для агрегации статистики и `TransactionMonitor`, имеющий структуру аппликативного функционального.

Использование моноидов позволило естественно комбинировать финансовые отчеты, функторы обеспечили отображение полученной информации, не изменяли при этом сам механизм проверок логики. Аппликативные функторы дали возможность построения сложных мониторов из простых проверок. Система включает два ключевых компонента: `FinancialSummary` с поддержкой моноидных операций для агрегации отчетов и `TransactionMonitor`.

Разработанное решение служит практическим примером применения абстракций функционального программирования для создания систем обработки данных в финансовой области.

Список литературы

- [1] Haskell Official Documentation – официальная документация по языку Haskell [Электронный ресурс]. <https://www.haskell.org/documentation/>
- [2] Digital Payment Systems and Transaction Processing / A. Rodriguez, S. Kumar // ACM Computing Surveys. — 2022. — Vol. 55, No. 8. — P. 1-38
https://www.researchgate.net/publication/364835418_Towards_the_Advancement_of_Cashless_Transaction_A_Security_Analysis_of_Electronic_Payment_Systems
- [3] Data Modeling for Financial Transactions: Best Practices and Patterns / K. Chen // Journal of Financial Data Science. — 2021. — Vol. 3, No. 2. — P. 45-62
https://www.researchgate.net/publication/385025548_Predictive_Analytics_in_Financial_Management_Enhancing_Decision-Making_and_Risk_Management

Приложение. Исходный код

Листинг 6: Полный исходный код программы

```
1  {-# LANGUAGE GeneralizedNewtypeDeriving #-}
2
3  module Lib where
4
5  data TransactionType = Income | Expense | Transfer
6  deriving (Show, Eq, Enum, Bounded)
7
8  data Transaction = Transaction
9  { transactionId :: Int
10   , amount :: Double
11   , transactionType :: TransactionType
12   , description :: String
13   , account :: String
14   }
15  deriving (Show, Eq)
16
17
18  data FinancialSummary = FinancialSummary
19  { totalIncome :: Double
20   , totalExpense :: Double
21   , transactionCount :: Int
22   , maxTransaction :: Maybe Transaction
23   }
24  deriving (Show, Eq)
25
26
27  instance Semigroup FinancialSummary where
28  (FinancialSummary inc1 exp1 cnt1 max1) <> (FinancialSummary inc2 exp2 cnt2 max2) =
29  FinancialSummary
30  { totalIncome = inc1 + inc2
31   , totalExpense = exp1 + exp2
32   , transactionCount = cnt1 + cnt2
33   , maxTransaction = case (max1, max2) of
34   (Nothing, Nothing) -> Nothing
35   (Just t1, Nothing) -> Just t1
36   (Nothing, Just t2) -> Just t2
37   (Just t1, Just t2) ->
38   if amount t1 >= amount t2
39   then Just t1
40   else Just t2
41  }
42
43  instance Monoid FinancialSummary where
44  mempty = FinancialSummary
45  { totalIncome = 0
46   , totalExpense = 0
47   , transactionCount = 0
48   , maxTransaction = Nothing
49  }
50
51  newtype TransactionMonitor a = TransactionMonitor
52  { runMonitor :: Transaction -> (Bool, a)
53  }
54
55  instance Functor TransactionMonitor where
56  fmap f (TransactionMonitor monitor) = TransactionMonitor $ \t ->
```

```

57 let (needsAttention, result) = monitor t
58 in (needsAttention, f result)
59
60 instance Applicative TransactionMonitor where
61 pure x = TransactionMonitor $ \_ -> (False, x)
62
63 (TransactionMonitor f) <*> (TransactionMonitor g) =
64 TransactionMonitor $ \t ->
65 let (needsAttention1, func) = f t
66 (needsAttention2, value) = g t
67 in (needsAttention1 || needsAttention2, func value)
68
69 transactionToSummary :: Transaction -> FinancialSummary
70 transactionToSummary t = FinancialSummary
71 { totalIncome = case transactionType t of Income -> amount t; _ -> 0
72   , totalExpense = case transactionType t of Expense -> amount t; _ -> 0
73   , transactionCount = 1
74   , maxTransaction = Just t
75 }
76
77 transactionsToSummary :: [Transaction] -> FinancialSummary
78 transactionsToSummary = foldMap transactionToSummary
79
80 combinePeriodReports :: [FinancialSummary] -> FinancialSummary
81 combinePeriodReports = mconcat
82
83 largeTransactionMonitor :: TransactionMonitor Double
84 largeTransactionMonitor = TransactionMonitor $ \t ->
85 let needsAttention = amount t > 1000
86 in (needsAttention, amount t)
87
88 suspiciousTransactionMonitor :: TransactionMonitor String
89 suspiciousTransactionMonitor = TransactionMonitor $ \t ->
90 let needsAttention = amount t < 0
91 in (needsAttention, description t)
92
93 combinedMonitor :: TransactionMonitor (Double, String)
94 combinedMonitor = (,) <$> largeTransactionMonitor <*> suspiciousTransactionMonitor
95
96 generateNotificationDouble :: (Bool, Double) -> Maybe String
97 generateNotificationDouble (needsAttention, amt)
98 | needsAttention = Just $ "␣" ++ show amt
99 | otherwise = Nothing
100
101 generateNotificationString :: (Bool, String) -> Maybe String
102 generateNotificationString (needsAttention, desc)
103 | needsAttention = Just $ "'" ++ desc ++ "'"
104 | otherwise = Nothing
105
106 generateNotificationPair :: (Bool, (Double, String)) -> Maybe String
107 generateNotificationPair (needsAttention, (amt, desc))
108 | needsAttention = Just $ "␣" ++ desc ++ "␣" ++ show amt
109 | otherwise = Nothing
110
111 complexMonitoringSystem :: Transaction -> [Maybe String]
112 complexMonitoringSystem t =
113 [ generateNotificationDouble $ runMonitor largeTransactionMonitor t
114 , generateNotificationString $ runMonitor suspiciousTransactionMonitor t
115 , generateNotificationPair $ runMonitor combinedMonitor t
116 ]

```